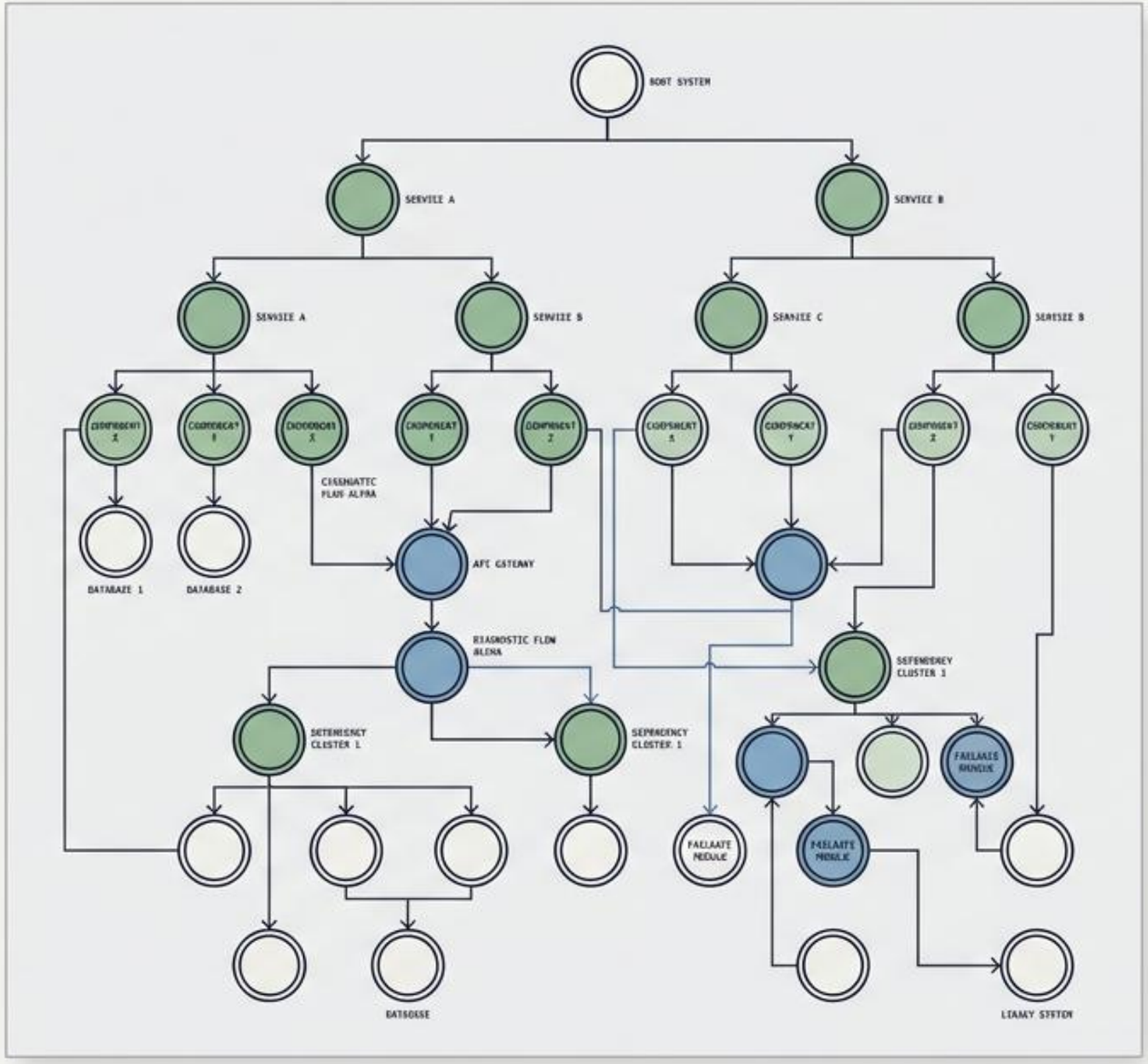
Dependency Knowledge Graphs

A definitive guide to modeling, analyzing, and diagnosing complex systemic dependencies.

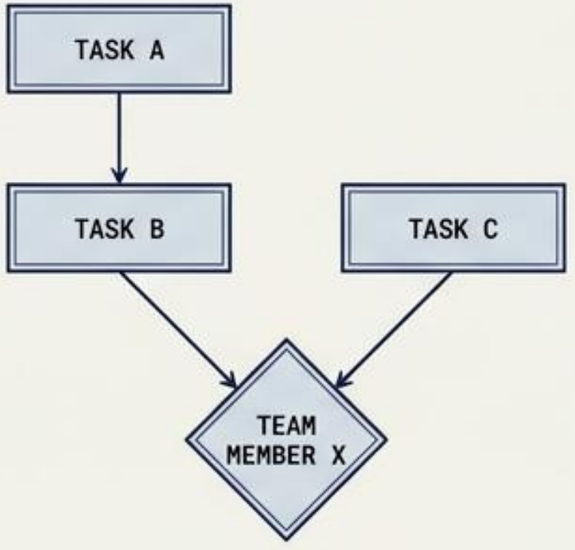
dr Zbigniew Galar



The Universal Challenge of Transitive Dependencies

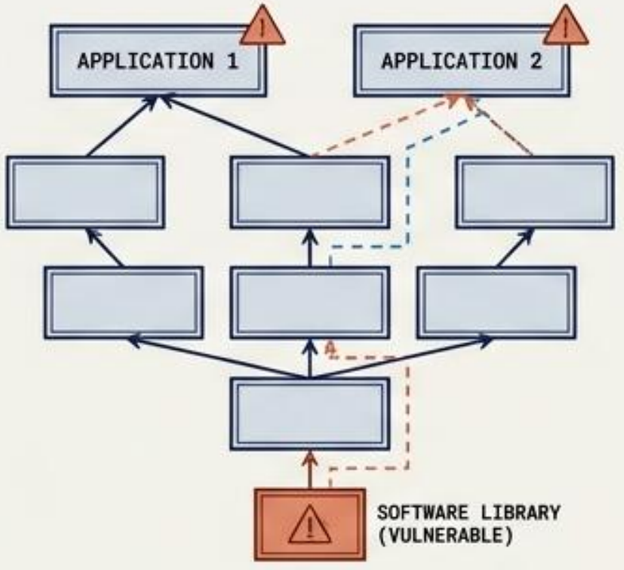
Project Management

Task B depends on Task A. Task B and C both depend on Team Member X.



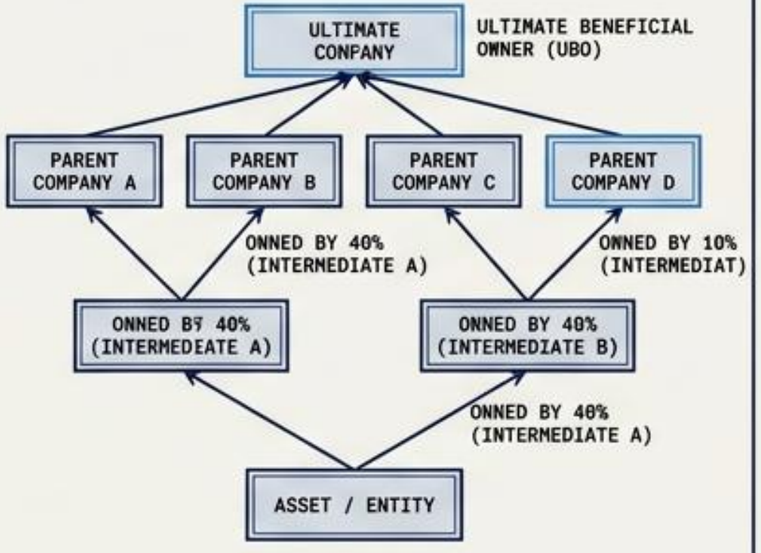
Cyber Security

A vulnerability hits a popular software library. Are your projects indirectly using it?



Finance & Compliance

AML checks require finding the Ultimate Beneficial Owner (UBO) through deep ownership chains.

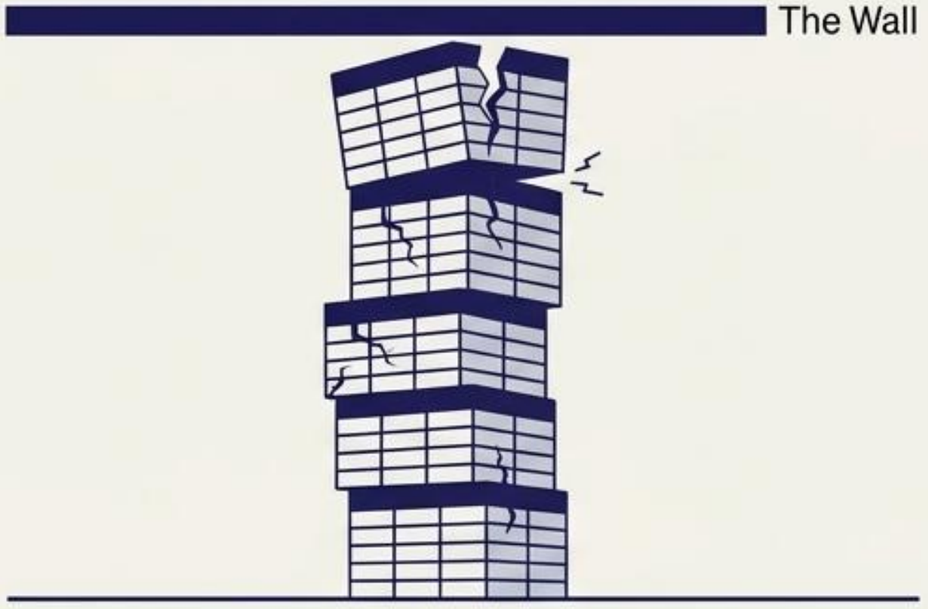


The common thread: individual direct dependencies form deep networks. Resolving them requires arbitrary-depth path analysis.

The Paradigm Shift: Breaking the Recursive Wall

The Tabular Limitation

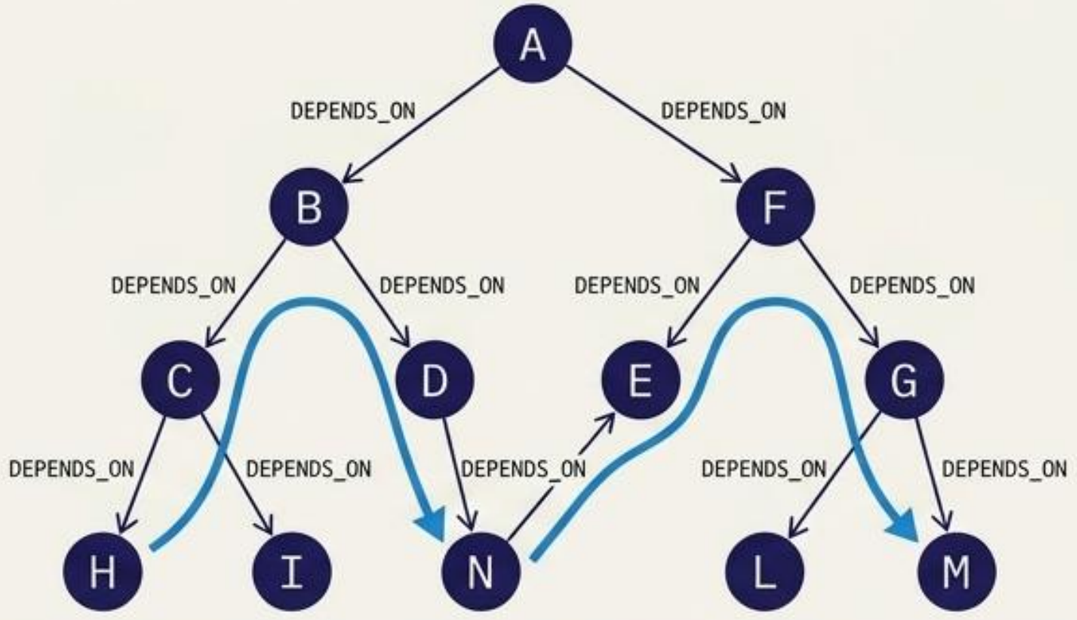
Relational technology wasn't designed for recursive path analysis.
Finding a hidden dependency requires explicit, pre-defined depth joins.




```
INNER JOIN dependencies d1
   ON d.depends_on = d1.elem
... [..]
```

The Graph Advantage

Knowledge graphs natively execute variable-length expressions.
A single query explores an arbitrary depth instantly.



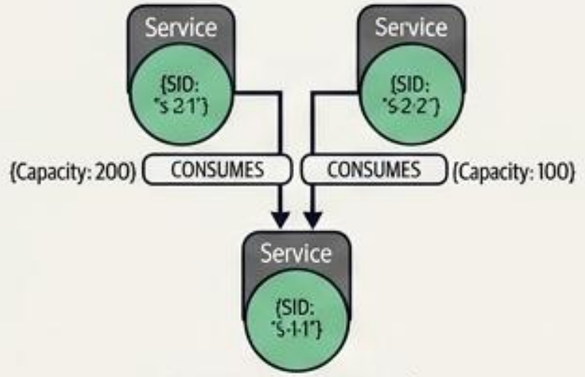

```
MATCH path = (:Element
       { id: 'A'})-[:DEPENDS_ON*]->(:Element { id: 'F'})
```

Graph pattern matching drastically lowers query complexity, yielding faster insights and lower maintenance costs.

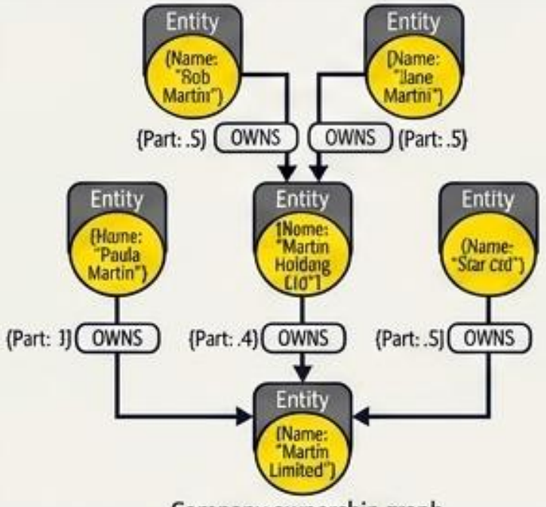
Micro Mechanics: Qualifying the Connections

Strength & Capacity

Dependencies are rarely binary. The property graph encodes the strength of a dependency directly on the relationship as a composable numeric value.



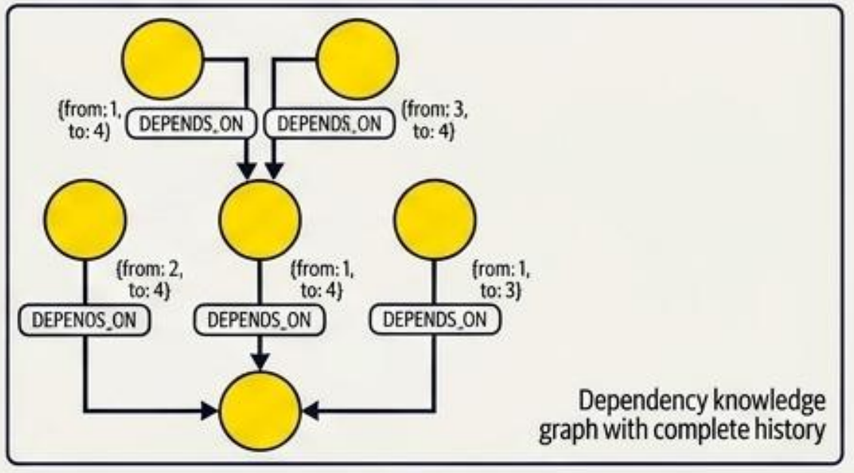
Capacity utilization graph



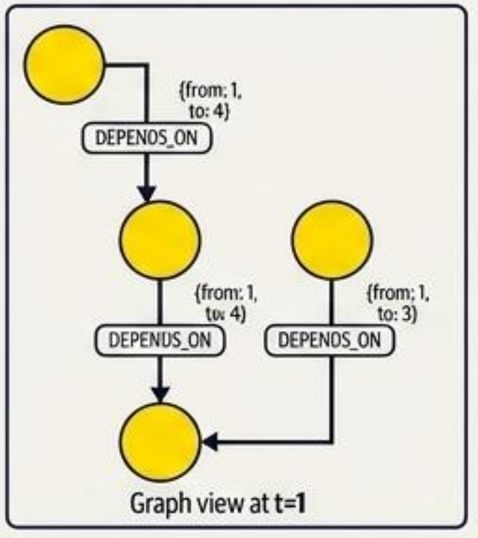
Company ownership graph

Temporal Validity

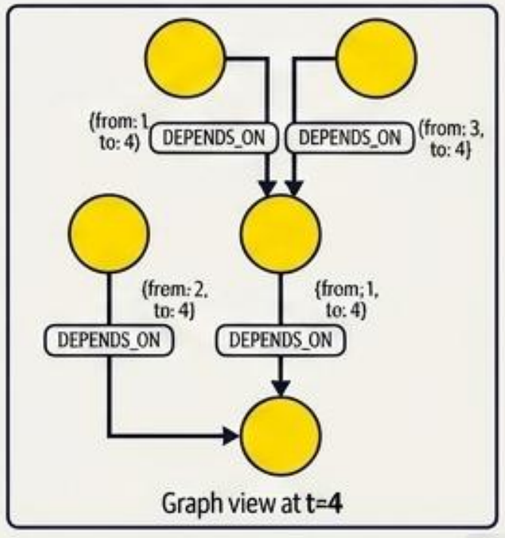
Capturing state over time. Queries are contextualized by passing a timestamp parameter to explore only active relationships.



Dependency knowledge graph with complete history

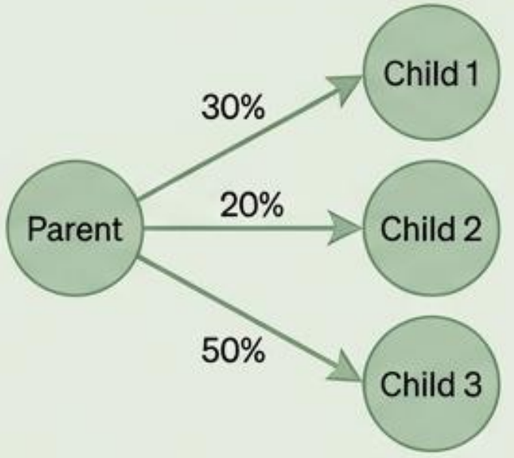
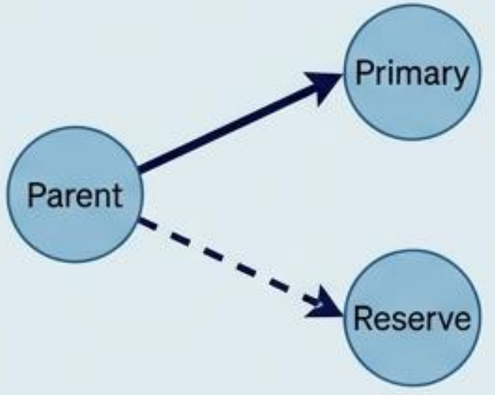


Graph view at t=1



Graph view at t=4

The Multidependency Matrix: Interpreting Complex Logic

Dependency Type	Logic & Use Cases	Arithmetic Rule	Visual Structure
Aggregate (Additive)	"Simultaneously Active." Spread over multiple entities based on weight. Use cases: Portfolio asset management, bonded communication links.	`parent_impact = child1_impact * weight + child2_impact * weight`	
Redundant (Protection)	"At Least One." Provides fault tolerance. Alternates are held in reserve. Use cases: Highly available (HA) architectures, alternative supply supply chain routing.	`parent_impact = min(child1_impact, child2_impact)`	

Impact Propagation: Calculating Aggregate Failures

How negative impacts cascade through additive systems

Step 1: The Observation

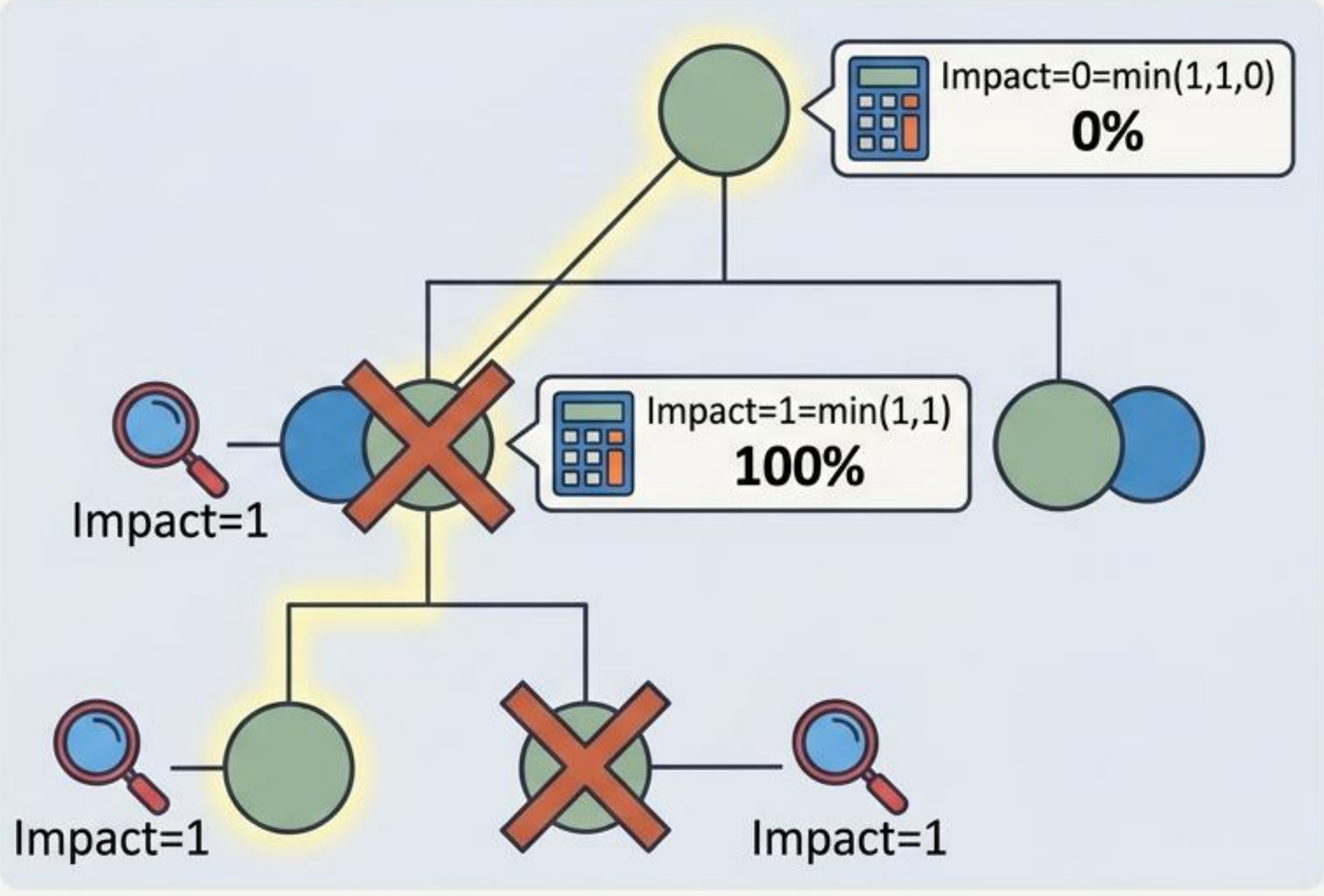
A raw impact is detected at the leaf node (e.g., hard drive fails, asset drops). Recorded as Impact = 1.

Step 2: The Calculation

Impact travels up the `[:DEPENDS_ON {mode:'AGG'}]` relationship. The parent node computes damage limited to the element's proportional contribution.

Step 3: The Result

The parent node degrades proportionally.

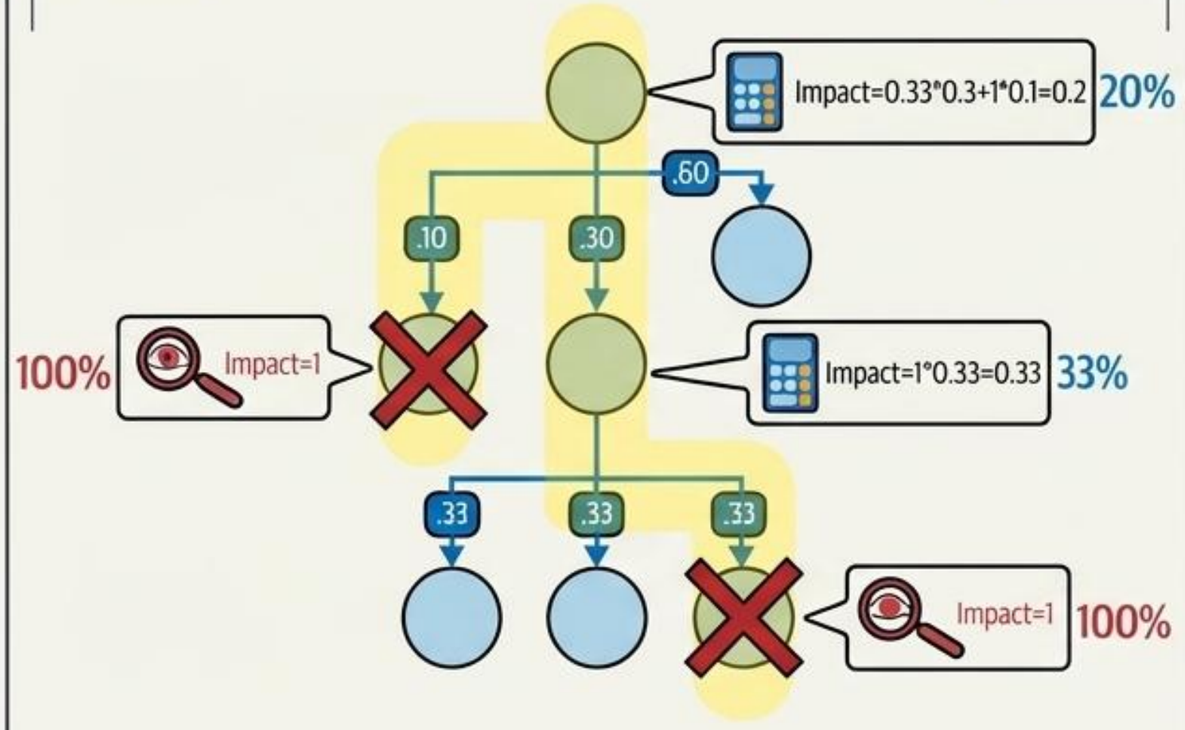


Impact Propagation: Calculating Redundant Failures

The Fault Absorber

In redundant structures, “protection nodes” absorb failures. If a preferred route fails, reserve capacity engages.

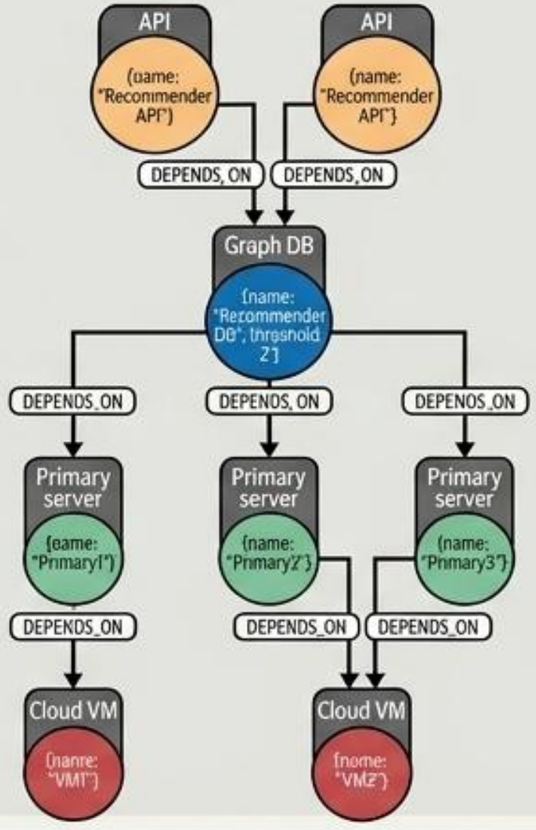
```
parent_impact = min(child1, child2)
```



Threshold Logic

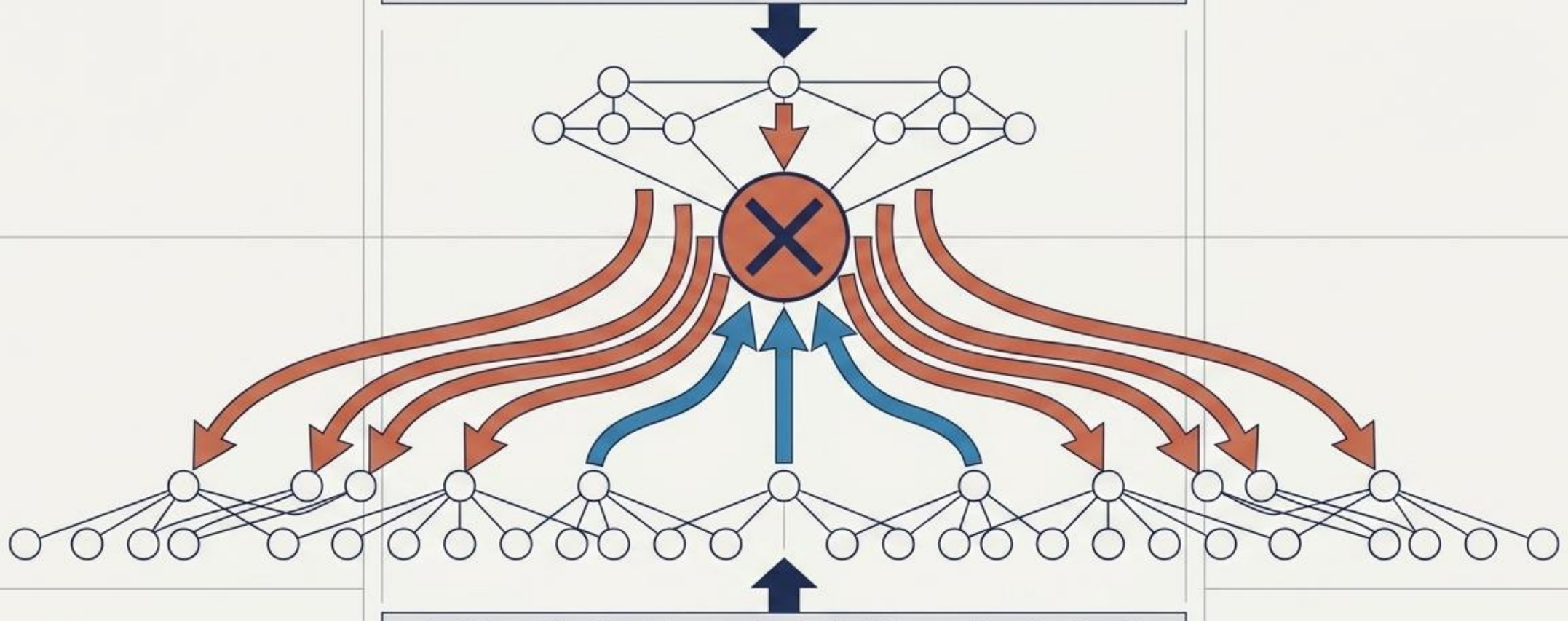
In complex architectures, a parent requires a minimum quorum of dependent entities to operate.

```
0 if sum((1 - child1), (1 - child2)...) >= threshold
else 1
```



The Analytical Symmetry: Top-Down vs. Bottom-Up

Top-Down: Impact Propagation (Predicting the Future)
Given a known failure, what is the topological 'blast radius'?
Used for real-time event enrichment.



Bottom-Up: Root Cause Analysis (Diagnosing the Past)
Given a chaotic stream of system alarms, what is the single common ancestor? Used for event correlation.

Advanced Analytics: Detecting Single Points of Failure (SPOF)

The Illusion of Fault Tolerance

Deploying multiple instances of fault-tolerant software is useless if they rely on the same underlying hardware, power source, or cooling system.

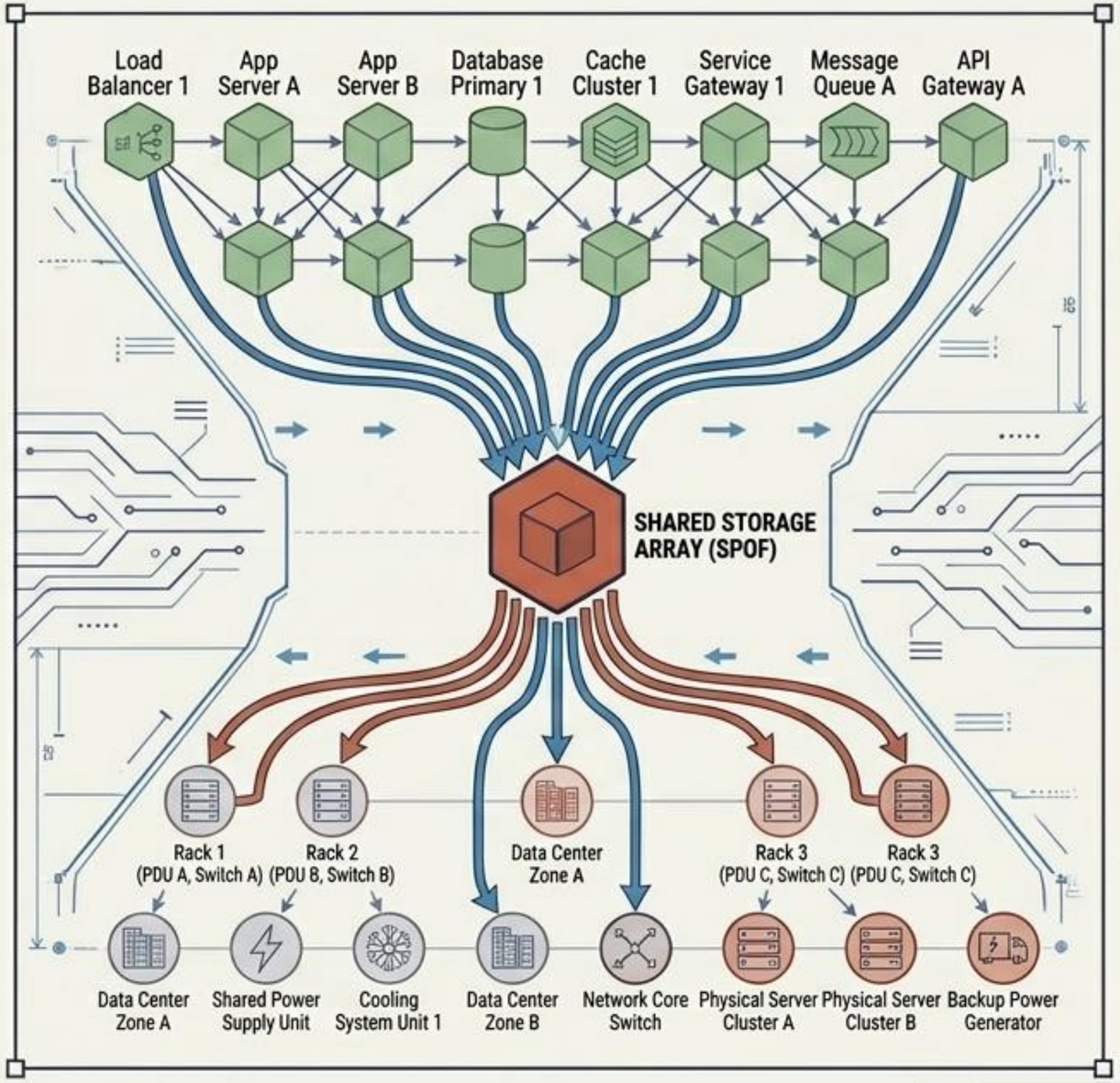
Graph Detection

A SPOF is simply a specific graph pattern: multiple variable-length dependency chains converging on a single element.

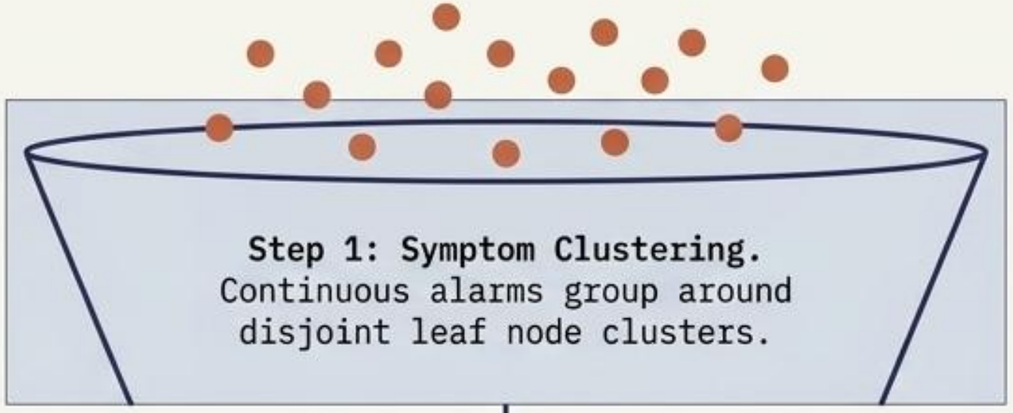
Business Value

Detect geographic bottlenecks in networks or shared VM dependencies before they trigger cascading system death.

```
MATCH alertPath = (spof)<-[:DEPENDS_ON*]-
  (e:Element)-[:DEPENDS_ON*]->(spof)
```



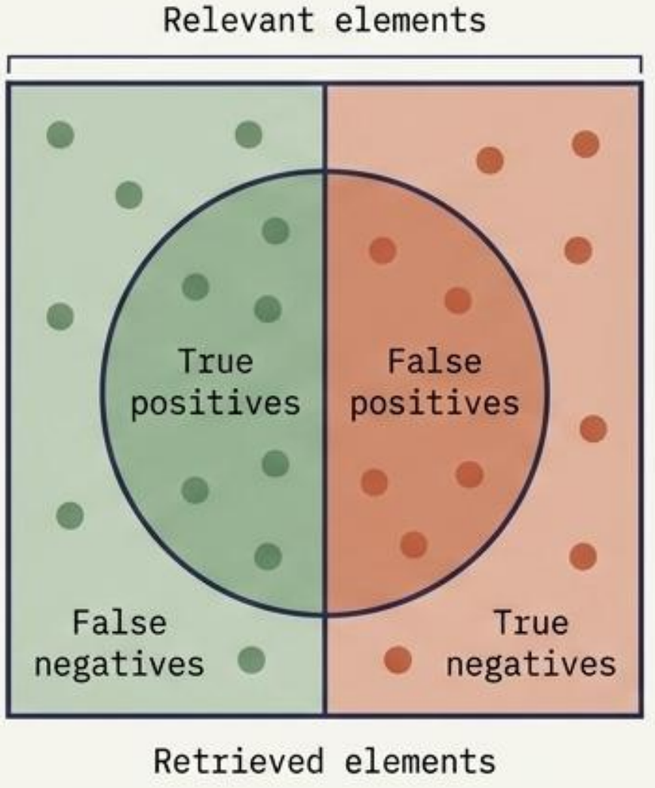
Advanced Analytics: The Root Cause Analysis Funnel



Step 1: Symptom Clustering.
Continuous alarms group around disjoint leaf node clusters.

Step 2: Statistical Ranking.
Apply Information Retrieval metrics to rank candidate nodes.

Output:
The True Culprit based on highest F-Score.



Precision= 

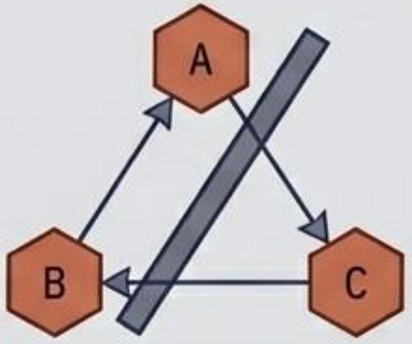
Recall= 

F-Score: The harmonic mean of Precision and Recall. Isolates the single most probable root cause.

System Integrity: The Dependency Validation Framework

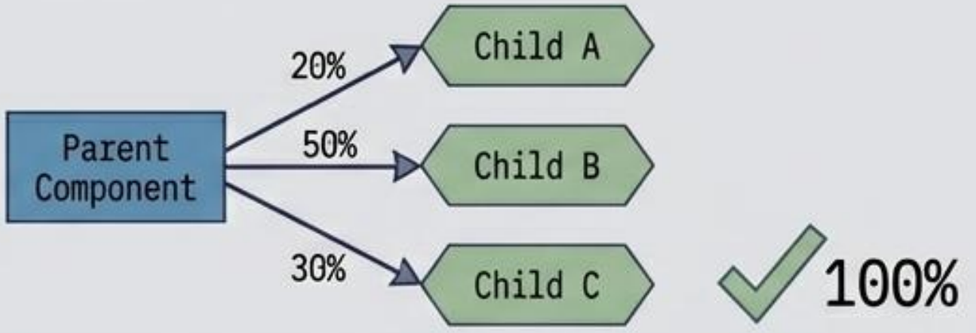
Rule 1: No Cycles (DAG Validation)

A dependency graph must be a Directed Acyclic Graph. A cycle means a component depends on itself—a logical domain bug.



Rule 2: Aggregates Equal Totals

In additive models, the relative weights of component dependencies must sum to exactly 1 (or 100%).

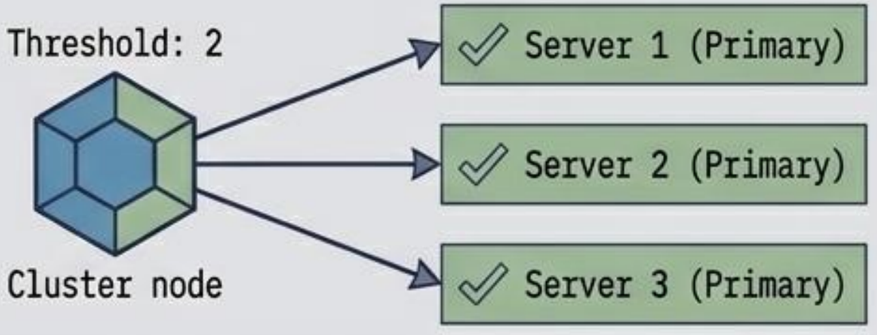


Rule 3: Consumption Cannot Exceed Production

Incoming and outgoing dependencies must balance.
If percentage_used > 100%, the graph is malformed.

Rule 4: Redundancy Aligns with Thresholds

A redundant setup must physically possess enough members to meet its threshold.



Enterprise Proof: The Vanguard Group

The Context

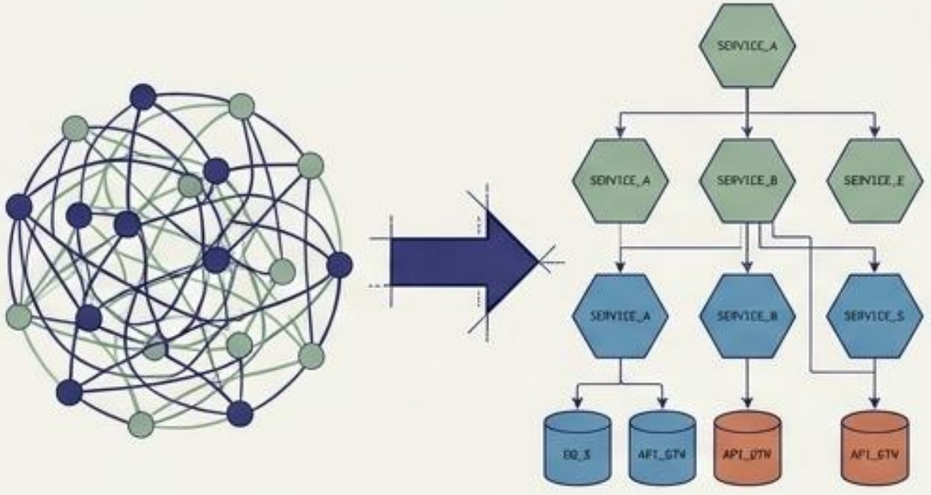
4,000,000

Lines of legacy code per module.

Vanguard needed to migrate legacy, monolithic Java systems into a microservices architecture. Manual interdependency mapping was impossible.

The Solution

Automated construction of a dependency knowledge graph during the CI/CD build process, mapping all code artifacts and architectural diagrams.



Legacy Monolith

Microservices Blueprint

The Impact

- Achieved total visibility into monolithic interdependencies.
- Pruned massive amounts of dead code to reduce technical debt.
- Detected misalignments between physical code and theoretical architecture.
- Generated code quality scorecards constraining service-to-service calls.